

SPRAWOZDANIE Z PROJEKTU

Uczelnia: Politechnika Krakowska

Kierunek: Informatyka

Przedmiot: Algorytmy i Struktury Danych

Autor: Kamil Gwozdek

Temat projektu: Temat 3. Kolejka priorytetowa

1. Opis problemu

Zasadniczym problemem poruszonym w projekcie jest efektywne zarządzanie dynamicznie zmieniającym się zbiorem danych, w którym każdy element posiada określoną wagę (priorytet), a kluczową operacją jest ciągle pobieranie elementu o najwyższym priorytecie (w mojej implementacji najwyższy priorytet = najniższa wartość). Tradycyjne struktury danych oparte na logice FIFO (First-In, First-Out) są niewystarczające, ponieważ ignorują ważność poszczególnych elementów. Z kolei każdorazowe sortowanie całej struktury po dodaniu nowego elementu jest wysoce nieefektywne czasowo.

Celem projektu było zaprojektowanie, zaimplementowanie w języku C++ oraz zbadanie wydajności algorytmów obsługujących kolejkę priorytetową, ze szczególnym uwzględnieniem zestawienia naiwnej struktury (listy jednokierunkowej) ze strukturą zoptymalizowaną (kopcem minimalnym - Min-Heap).

Aby nadać projektowi praktyczny kontekst, problem zwizualizowałem na przykładzie priorytetyzacji pakietów sieciowych (gdzie najwyższy priorytet ma np. pakiet głosowy komunikatora Discord, a najniższy przesyłanie plików w tle).

2. Omówienie różnych metod rozwiązania problemu

Problem wyciągania wartości skrajnych (minimum lub maksimum) z dynamicznego zbioru można rozwiązać za pomocą różnych struktur danych. W ramach projektu zbadałem dwa fundamentalne podejścia:

- **Implementacja listowa:** Opiera się na liście jednokierunkowej (nieposortowanej). Nowe elementy są po prostu dopinane na koniec struktury. Zaletą tej metody jest szybkie wstawianie elementów. Główną wadą jest drastyczny spadek wydajności przy operacji usuwania - aby znaleźć element o najmniejszym kluczu (najwyższym priorytecie), algorytm musi dokonać pełnego, liniowego przeszukania całej struktury, co przy dużej liczbie danych obciąża procesor i po prostu zajmuje dużo czasu.
- **Implementacja za pomocą kopca (Min-Heap):** Wykorzystuje strukturę kompletnego drzewa binarnego (reprezentowanego najczęściej w postaci tablicy). Kopiec minimalny gwarantuje, że wartość węzła-rodzica jest zawsze mniejsza lub równa wartościom jego dzieci. Dzięki temu element o najwyższym priorytecie (minimum) zawsze znajduje się w

korzeniu drzewa. Przywracanie własności kopca po operacjach modyfikacji odbywa się jedynie wzdłuż wysokości danej gałęzi drzewa, co daje gwarancję bardzo szybkiego czasu wykonania - logarytmicznego w stosunku do liczby elementów.

3. Przykłady zastosowania

Struktura kolejki priorytetowej jest kluczowym elementem wielu zaawansowanych algorytmów i systemów informatycznych. Przykłady obejmują:

- **Algorytmy grafowe:** Kolejka priorytetowa jest fundamentem algorytmu Dijkstry (szukanie najkrótszej ścieżki w grafie) oraz algorytmu Prima (wyznaczanie minimalnego drzewa rozpinającego).
- **Systemy operacyjne:** Planista (scheduler) wykorzystuje priorytety do zarządzania procesami i wątkami, wyłaszczając zadania mało istotne na rzecz procesów systemowych.
- **Mechanizmy QoS (Quality of Service) w sieciach komputerowych:** Przydzielanie pierwszeństwa wrażliwym na opóźnienia pakietom sieciowym (VoIP, wideo) przed zwykłym transferem danych. Ten właśnie przypadek użycia stanowi praktyczny kontekst do niniejszego projektu, co zweryfikowałem dodatkowo symulacją logiczną na prostej sieci i komunikacji między siecią lokalną, a serwerem dostarczającym usługi w środowisku Cisco Packet Tracer.

4. Opis zastosowanych algorytmów

Pełna dokumentacja aplikacji dostępna pod adresem:

<https://noksiaczek.github.io/QoS-Queue-Analyzer/html/index.html>

W projekcie zaimplementowałem klasy w C++ obsługujące cztery kluczowe operacje dla obu badanych struktur danych.

1. **Wstaw (Insert):**
 - *Lista:* Alokacja nowego węzła i wpięcie go na koniec listy przy pomocy wskaźnika *tail*. Czas operacji: $O(1)$.
 - *Kopiec:* Wstawienie elementu na koniec tablicy (ostatni liść w drzewie) i wywołanie algorytmu *heapifyUp*, który w pętli porównuje dodany węzeł z jego rodzicem i zamienia je miejscami, aż do przywrócenia własności kopca. Czas operacji: $O(\log N)$.
2. **Znajdź wartość minimalną (front/find):**
 - *Lista:* Iteracja po wszystkich węzłach za pomocą wskaźnika pomocniczego i zapisywanie najmniejszej napotkanej wartości. Czas operacji: $O(N)$.
 - *Kopiec:* Zwrócenie wartości z korzenia (indeks 0 w tablicy). Czas operacji: $O(1)$.
3. **Usuń wartość minimalną (pop):**
 - *Lista:* Po znalezieniu minimum (jak wyżej), algorytm przepina wskaźnik *next* poprzednika i zwalnia pamięć usuwanego węzła. Czas operacji: w najgorszym przypadku $O(N)$.

- *Kopiec*: Zapisanie wartości z korzenia, nadpisanie korzenia wartością z ostatniego liścia drzewa, usunięcie tego liścia i wywołanie algorytmu *heapifyDown*, który przesuwa nowy korzeń w dół struktury do odpowiedniej pozycji. Czas operacji: $O(\log N)$.
4. **Zamień klucz na większy (Increase Key):**
- *Kopiec*: Operacja odszukania węzła po identyfikatorze, zwiększenia jego priorytetu liczbowego (co oznacza spadek jego ważności) i wywołania dla niego procedury *heapifyDown*, która "zepchnie" go na niższą warstwę drzewa w czasie $O(\log N)$.

5. Ilustracja działania programu

Projekt składa się z napisanej w C++ aplikacji konsolowej oraz skryptów w języku Python do ewaluacji matematycznej.

A. Działanie algorytmiczne w konsoli:

Aplikacja w C++ na bieżąco prezentuje w konsoli graficzną reprezentację struktury kopca za pomocą znaków ASCII. Umożliwia prześledzenie metod *heapifyUp* oraz *heapifyDown* krok po kroku po wstawieniu lub usunięciu węzła.

```
=====
FAZA 1: ODBIERANIE RUCHU SIECIOWEGO
Wpadl pakiet: [5] Wyszukiwanie w Wikipedii
=====

      -> [5]
    -> [1]
      -> [3]
-> [1]    -> [7]
      -> [2]
          -> [5]
          -> [4]
              -> [9]
```

```
=====
FAZA 2: QOS WYPUSZCZA NAJWAZNIEJSZY PAKIET
Wyslano: Prio 2 | League of Legends
=====

      -> [5]
          -> [7]
-> [3]    -> [5]
          -> [4]
              -> [9]
```

B. Badanie złożoności obliczeniowej (Benchmark):

Zasadniczym celem była analiza wydajności. Program w C++ przeprowadza testy obciążeniowe, wstawiając i usuwając zbiory danych o rozmiarach $N \in [1000, 100000]$ (generowane losowo). Wyniki w mikrosekundach wyeksportowano do pliku CSV i zwizualizowano na wykresach w Pythonie.



Wnioski z wykresów: Teoretyczne założenia matematyczne zostały potwierdzone. O ile wstawianie elementów w liście jest minimalnie szybsze niż w kopcu, o tyle wypuszczanie elementów drastycznie pogarsza wydajność listy, prowadząc do kwadratowej złożoności czasowej $O(N^2)$ dla całego procesu. Kopiec minimalny działa w przewidywalnym czasie logarytmiczno-liniowym $O(N * \log N)$, wykonując to samo zadanie dla 100 tysięcy pakietów niemal natychmiastowo. Dowodzi to przewagę zoptymalizowanych struktur danych w obsłudze dużych i dynamicznych zbiorów.